Bonus (optional): the heap, and proofs without a bound

Deadline: **December 7, 2026**. Write your answers in ANSWERS.md.

1. Composite: a data structure on the heap

`Composite.java` is a tree in which every node caches the maximum value of its subtree (`value`, `maxChild`), and `addChild` updates the caches of the ancestors. It comes from a verification benchmark (AutoProof's `composite.e`, through the Java version used in research on assertion inference).

`CompositeCheck.addChildren` lets JBMC build trees of 3 nodes with any sequence of `addChild` calls allowed by its precondition, and checks the class invariant of every node after each call. So far it checks only one part of the invariant.

Complete `checkInvariant` from the comments on the fields of `Composite`, and run `./bmc CompositeCheck.addChildren --unwind K --no-unwinding-assertions` with K = 2, 3, 4 and 5.

- **Q** Which part of the invariant breaks, after which calls? What is wrong in `update`? Fix it and check again.
- **Q** The bug needs only two calls to `addChild`. Why does JBMC need K = 4 to find it?
- **Q** JBMC's input here is a heap: objects, references between them, aliasing. How large is the formula compared with the previous exercises? (Compare K = 4 and 5: with K < 4 every execution is cut and JBMC builds no formula.) What would you need to verify `Composite` with OpenJML?

★ The original requires `c` to be a single node. Allow subtrees: drop `c.firstChild == null` from `addChild` and from the harness's `assume`, and run again. Does your invariant notice what goes wrong? If not, add the missing property, then fix `addChild` so that subtrees can be added safely.

2. SafeLock for any number of events

In homework plus (exercise 5), JBMC checked `SafeLock` for sequences of up to K events. This exercise works on the same file, `plus/SafeLock.java`, with your fix. Run `./esc plus/SafeLock.java`.

- **Q** Why does OpenJML think `failed++` may overflow?

Write a class invariant (`//@ private invariant ...;`) relating `failed` and `state` that makes `SafeLock` verify. Then put the bug of exercise 5 back.

- **Q** Does your invariant catch it? What does this proof give you that JBMC did not?

Fix the bug again before you submit: the committed `plus/SafeLock.java` must verify with your invariant.

Before you submit

- `./bmc CompositeCheck.addChildren --unwind 5 --no-unwinding-assertions` gives VERIFICATION SUCCESSFUL with your complete invariant and the fixed `update`.
- `./esc plus/SafeLock.java` says OK with your class invariant.
- Every question in `ANSWERS.md` is answered.